SIMATS ENGINEERING

ASSESSMENT TOOL 2

DATA INTERPRETATION

COURSE NAME: INTERNET PROGRAMMING **Name:** T. Thanmai Reddy

COURSE CODE: CSA4303 **Reg. No.:** 192425372

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1.Identify any incorrect or invalid fields in the header.

2. Analyze the issue with Content-Length and explain its impact.

3. Determine whether the status code matches the response correctness.

4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
 <head>
 <title>TestPage</title>
 </head>
 <body>
 <div>
 <p>HelloWorld
 </div>
 </body>
```

```
</html>
```

Questions:

5.Identify missing or improperly closed elements.

6. Analyze how the browser will interpret this structure.

7. Determine the impact on rendering and layout.

8. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL / Data Transmission	Data Visibility / Security Level
GET	/login?user=admin&pass=1234	Visible in URL; low security
POST	/login (data in request body)	Not shown in URL; moderate security

9.Interpret the differences in data transmission between GET and POST.

10. Analyze which method is more suitable for login and why.

11. Identify security risks in using GET for authentication.

12. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
 <title>Sample</title>
</head>
<body>
 <h1>Welcome
 <p>This is a paragraph
 <img src="image.jpg">
</body>
</html>
```

Questions:

13. Identify missing closing tags.

14. Analyze issues with the <img> tag.

15. Determine how browsers will handle these errors.

16. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser.
2. DNS lookup occurs.
3. HTTP request is sent.
4. Server processes request.
5. Response is returned.
6. Browser renders page.

Questions:

17. Interpret each step in the communication flow.
18. Identify where delays can occur.
19. Analyze the role of DNS in the process.
20. Suggest optimizations to improve performance.

Code of Conduct

I ___T. Thanmai Reddy / 192425372_____ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Data	23/25	Accurately interprets all data patterns, trends, and relationships.	Interprets data correctly with minor gaps.	Partial understanding; misses key trends.	Misinterprets or fails to understand data.
Analysis	22/25	Deep analysis with strong logical reasoning and justification.	Reasonable analysis with some justification.	Basic analysis with weak reasoning.	No meaningful analysis provided.
Application of Concepts	18/20	Effectively applies relevant concepts/tools accurately.	Applies concepts with minor errors.	Limited application; requires guidance.	Unable to apply concepts to data.
Conclusion	18/20	Clear, meaningful conclusions with strong insights.	Valid conclusions with moderate insights.	Conclusions are vague or weak.	No clear or valid conclusions.
Clarity	9/10	Clear structure, visuals, and terminology.	Generally clear with minor issues.	Lacks clarity or proper structure.	Poor presentation; difficult to understand.

Total:90/100

ANSWERS

1. HTTP Response Analysis

1.1 Identify Incorrect or Invalid Fields in the Header

The response header contains a major error in the Content-Length field. The value is -150, but Content Length represents the number of bytes in the message body and cannot be negative. The Date field also appears inconsistent: 1 April 2026 was a Wednesday, not Monday. The date should therefore be corrected if it is intended to represent that calendar date.

1.2 Analyze the Content-Length Issue and Its Impact

Content-Length tells the client how many bytes of response content to expect. A negative value is invalid and does not describe a possible message-body length. It may cause a client, intermediary, or validator to reject or mishandle the response. The server should provide the actual non-negative byte length of the response body. If the response body is 150 bytes, the field should be Content-Length: 150.

1.3 Determine Whether the Status Code Matches the Response Correctness

HTTP/1.1 200 OK indicates that the request was successfully handled. However, a success status does not make malformed response headers valid. The server should correct the invalid header. If the server can return the requested HTML page successfully, 200 OK may remain appropriate; if it cannot fulfill the request, it should return a status code that reflects the actual outcome.

1.4 Suggest Corrections

An example of a corrected response header is shown below. The Content-Length value must match the actual number of bytes in the body; the value 150 is only an example.

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

- Use a valid, non-negative Content-Length that matches the exact response-body size.
- Use a correct HTTP date and weekday.
- Keep the status code consistent with the actual result of the request.
- Validate generated response headers before sending them to the client.

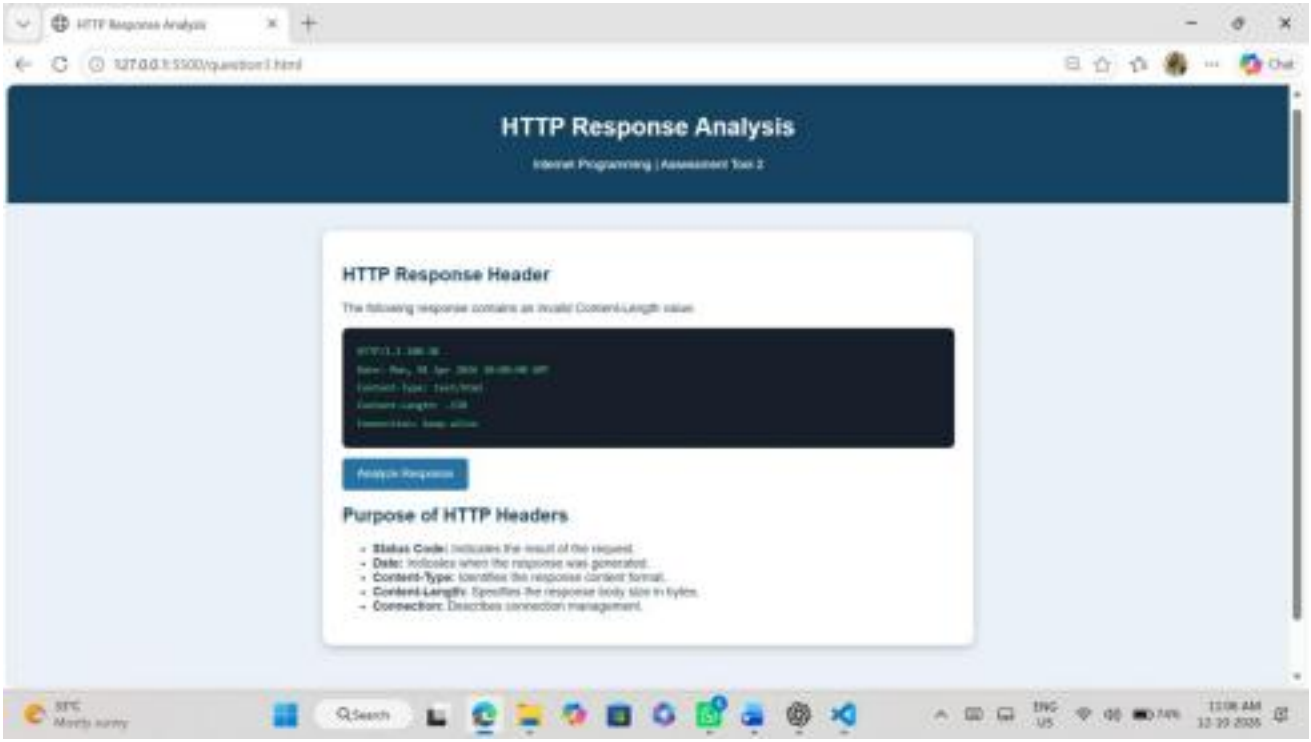
HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>HTTP Response Analysis</title>
 <style>
 * {
 box-sizing: border-box; }
 body {
 margin: 0;
 font-family: Arial, sans-serif;
 background: #eaf2f8;
 color: #2c3e50;}
 header {
 background: #154360;
 color: white;
 text-align: center;
 padding: 25px; }
 main {
 max-width: 850px;
 margin: 35px auto;
 padding: 25px;
 background: white;
 border-radius: 12px;
 box-shadow: 0 4px 12px #bdc3c7;}
 h2 {
 color: #154360; }
 pre {
 background: #17202a;
 color: #58d68d;
 padding: 20px;
 border-radius: 8px;
 overflow-x: auto;
 line-height: 1.8; }
 button {
 background: #2874a6;
```

```
color: white;
padding: 12px 20px;
border: none;
border-radius: 5px;
cursor: pointer;
font-size: 15px; }
button:hover {
background: #154360; }
.result {
margin-top: 20px;
padding: 18px;
border-radius: 8px;
display: none;
line-height: 1.8; }
.invalid {
background: #fadbd8;
border-left: 5px solid #c0392b; }
.valid {
background: #d5f5e3;
border-left: 5px solid #229954;}
footer {
text-align: center;
padding: 20px;}
</style>
</head>
<body>
<header>
<h1>HTTP Response Analysis</h1>
<p>Internet Programming | Assessment Tool 2</p>
</header>
<main>
<h2>HTTP Response Header</h2>
<p>The following response contains an invalid Content-Length value.</p>
<pre>HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
```

```
Connection: keep-alive</pre>
 <button onclick="analyzeHeader()">Analyze Response</button>
<div id="result" class="result"></div>
 <h2>Purpose of HTTP Headers</h2>
 <ul>
 <li><b>Status Code:</b> Indicates the result of the request.</li>
<li><b>Date:</b> Indicates when the response was generated.</li>
<li><b>Content-Type:</b> Identifies the response content format.</li>
<li><b>Content-Length:</b> Specifies the response body size in bytes.</li>
<li><b>Connection:</b> Describes connection management.</li>  </ul>
</main>
<footer>
 <p>SIMATS ENGINEERING | Internet Programming</p>
</footer>
<script>
 function analyzeHeader() {
 const result = document.getElementById("result");
 result.style.display = "block";
 result.className = "result invalid";
 result.innerHTML = `
 <h3>Analysis Results</h3>
 <p><b>1. Invalid Content-Length:</b>
 The value -150 is invalid because content length
 cannot be negative.</p>
 <p><b>2. Incorrect Date:</b>
 1 April 2026 was Wednesday, not Monday.</p>
 <p><b>3. Status Code:</b>
 200 OK indicates success, but it does not make
 invalid headers correct.</p>
 <p><b>4. Suggested Corrections:</b>
 Use a non-negative Content-Length matching the
 actual response body and correct the date.</p>
 `;
 }
</script>
</body>
</html>
```

Output: -



2. DOM Structure Analysis

2.1 Identify Missing or Improperly Closed Elements

The paragraph element <p> is opened but not explicitly closed before . The document also omits the <!DOCTYPE html> declaration and the lang attribute on the root <html> element. Modern HTML parsers can recover from some omissions, but explicitly writing a complete structure improves maintainability and standards compliance.

2.2 Analyze How the Browser Interprets the Structure

Browsers use an HTML parser that attempts to recover from common markup errors. When the parser reaches , it will generally close an open paragraph element as part of error recovery. The browser may still display the text HelloWorld, but the resulting DOM is created through parser-recovery rules rather than from clearly written markup.

2.3 Determine the Impact on Rendering and Layout

The page may appear nearly correct because browsers repair many common HTML errors automatically. However, invalid or unclear markup can lead to unexpected DOM structure, styling differences, and accessibility problems as the page becomes more complex. CSS selectors and scripts may also behave unexpectedly if the actual DOM differs from the author's intention.

2.4 Suggest a Corrected DOM Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>TestPage</title>
</head>
<body>
 <div>
 <p>HelloWorld</p>
 </div>
</body>
</html>
```

Explanation:

- <!DOCTYPE html> declares an HTML5 document.
- The lang attribute identifies the language of the page.
- The paragraph is explicitly closed before the containing div ends.
- The character-set and viewport metadata support reliable text rendering and responsive display.

HTML Code:

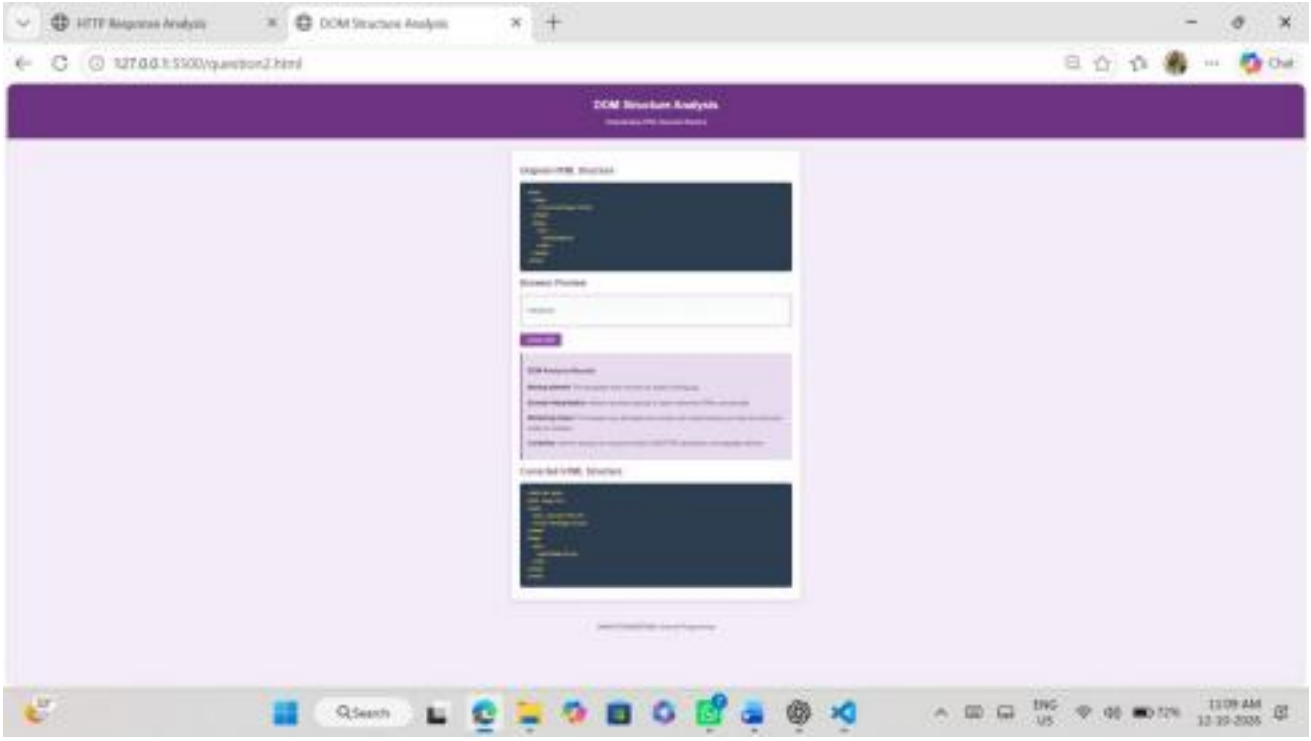
```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>HTTP Response Analysis</title>
 <style>
 * {
 box-sizing: border-box;
 }
 body {
 margin: 0;
 font-family: Arial, sans-serif;
background: #eaf2f8;
 color: #2c3e50;
 }
 header {
 background: #154360;
```

```
color: white;
text-align: center;
padding: 25px;
}
main {
max-width: 850px;
margin: 35px auto;
padding: 25px;
background: white;
border-radius: 12px;
box-shadow: 0 4px 12px #bdc3c7; }  h2 {
color: #154360;}
pre {
background: #17202a;
color: #58d68d;
padding: 20px;
border-radius: 8px;
overflow-x: auto;
line-height: 1.8; }
button {
background: #2874a6;
color: white;
padding: 12px 20px;
border: none;
border-radius: 5px;
cursor: pointer;
font-size: 15px; }
button:hover {
background: #154360; }
.result {
margin-top: 20px;
padding: 18px;
border-radius: 8px;
display: none;
line-height: 1.8; }
.invalid {
background: #fadbd8;
```

```
 border-left: 5px solid #c0392b; }
 .valid {
 background: #d5f5e3;
 border-left: 5px solid #229954; }
 footer {
 text-align: center;
 padding: 20px; }
 </style>
</head>
<body>
<header>
 <h1>HTTP Response Analysis</h1>
 <p>Internet Programming | Assessment Tool 2</p>
</header>
<main>
 <h2>HTTP Response Header</h2>
 <p>The following response contains an invalid Content-Length value.</p>
<pre>HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive</pre>
 <button onclick="analyzeHeader()">Analyze Response</button>
 <div id="result" class="result"></div>
 <h2>Purpose of HTTP Headers</h2>
 <ul>
 <li><b>Status Code:</b> Indicates the result of the request.</li>
<li><b>Date:</b> Indicates when the response was generated.</li>
<li><b>Content-Type:</b> Identifies the response content format.</li>
<li><b>Content-Length:</b> Specifies the response body size in bytes.</li>
<li><b>Connection:</b> Describes connection management.</li>  </ul>
</main>
<footer>
 <p>SIMATS ENGINEERING | Internet Programming</p>
</footer>
<script>
 function analyzeHeader() {
```

```
const result = document.getElementById("result");
result.style.display = "block";
result.className = "result invalid";
result.innerHTML = `
<h3>Analysis Results</h3>
<p><b>1. Invalid Content-Length:</b>
The value -150 is invalid because content length
cannot be negative.</p>
<p><b>2. Incorrect Date:</b>
1 April 2026 was Wednesday, not Monday.</p>
<p><b>3. Status Code:</b>
200 OK indicates success, but it does not make
invalid headers correct.</p>
<p><b>4. Suggested Corrections:</b>
Use a non-negative Content-Length matching the
actual response body and correct the date.</p>
`; }
</script>
</body>
</html>
```

Output: -



3. GET vs POST in Login System

3.1 Differences in Data Transmission

GET commonly sends form values as query parameters in the URL. This makes the values visible in the address bar and potentially in browser history, logs, copied links, and referrer data. POST sends submitted values in the request body instead of appending them to the URL. POST does not itself encrypt data; HTTPS is needed to protect information while it travels between the browser and server.

3.2 Suitable Method for Login

POST is generally more suitable for login credentials because it avoids placing usernames and passwords directly in the URL. The login endpoint should also use HTTPS, validate the submitted credentials on the server, and return appropriate success or failure responses.

3.3 Security Risks of GET for Authentication

- Credentials may remain in browser history or be copied when a URL is shared.
- URLs can be recorded in web-server, proxy, analytics, and monitoring logs.
- Sensitive query strings may be exposed through referrer information in some situations. • GET

requests may be cached or bookmarked, making it unsuitable for transmitting passwords. Using POST reduces URL exposure but does not make authentication secure by itself.

3.4 Best Practices for Secure Login

21. Use HTTPS for all login and authenticated pages.
22. Submit credentials with POST rather than placing them in query parameters. 23. Validate credentials on the server and store passwords using a suitable password-hashing algorithm.
24. Use secure session management and cookies with Secure, HttpOnly, and appropriate SameSite attributes.
25. Apply rate limiting and monitoring to reduce automated password-guessing attempts.
26. Avoid displaying sensitive information in URLs, logs, or error messages.

HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

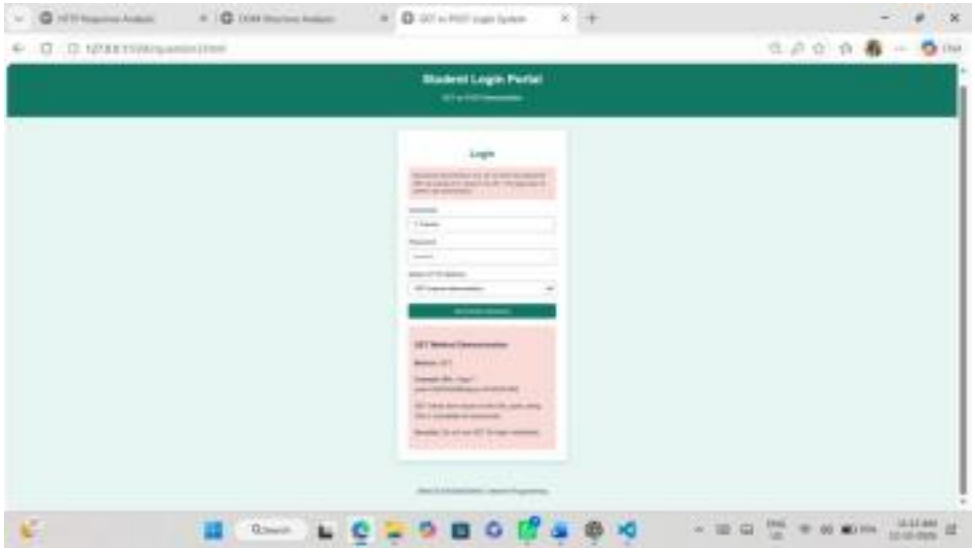
```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>GET vs POST Login System</title>
<style>
* {
box-sizing: border-box; }
body {
margin: 0;
font-family: Arial, sans-serif;
background: #e8f6f3;
color: #2c3e50;}
header {
background: #117864;
color: white;
text-align: center;
padding: 25px;}
main {
max-width: 450px;
margin: 35px auto;
padding: 30px;
background: white;
border-radius: 12px;
box-shadow: 0 4px 12px #a2d9ce; }  h2 {
text-align: center;
color: #117864; }
label {
display: block;
margin-top: 18px;
margin-bottom: 7px; }
input, select {
width: 100%;
padding: 12px;
border: 1px solid #aaa;
border-radius: 5px; }
button {
width: 100%;
margin-top: 20px;
```

```
padding: 12px;
background: #117864;
color: white;
border: none;
border-radius: 5px;
cursor: pointer; }
button:hover {
background: #0b5345; }
.result {
display: none;
margin-top: 20px;
padding: 15px;
background: #d5f5e3;
border-radius: 8px;
overflow-wrap: anywhere;
line-height: 1.7;}
.warning {
background: #fadbd8;
padding: 12px;
border-radius: 5px;
font-size: 13px;
line-height: 1.6; }
footer {
text-align: center;
padding: 20px; }
</style>
</head>
<body>
<header>
 <h1>Student Login Portal</h1>
 <p>GET vs POST Demonstration</p>
</header>
<main>
 <h2>Login</h2>
 <p class="warning">
 Educational demonstration only. Do not enter real passwords.
GET can expose form values in the URL. This page does not
```

```
perform real authentication.
 </p>
 <form id="loginForm">
 <label for="username">Username:</label>
 <input type="text" id="username" required>
 <label for="password">Password:</label>
 <input type="password" id="password" required>
<label for="method">Select HTTP Method:</label>
<select id="method">
 <option value="POST">POST</option>
 <option value="GET">GET (Insecure demonstration)</option>
</select>
 <button type="submit">Demonstrate Submission</button>
</form>
 <div id="result" class="result"></div>
</main>
<footer>
 <p>SIMATS ENGINEERING | Internet Programming</p>
</footer>
<script>
 document.getElementById("loginForm").addEventListener(
"submit",
 function(event) {
 event.preventDefault();
 const username =
 document.getElementById("username").value;  const
password =
 document.getElementById("password").value;  const
method =
 document.getElementById("method").value;  const
result =
 document.getElementById("result");
 result.style.display = "block";
 if (method === "GET") {
 result.style.background = "#fadbd8";
 result.innerHTML = `
 <h3>GET Method Demonstration</h3>
```

```
<p><b>Method:</b> GET</p>
 <p><b>Example URL:</b>
 /login?user=USERNAME&amp;amp;pass=PASSWORD</p>  <p>GET
sends form values in the URL query string.  This is unsuitable for
passwords.</p>
 <p><b>Security:</b> Do not use GET for login
credentials.</p>
 } else {
 result.style.background = "#d5f5e3";
 result.innerHTML = `
 <h3>POST Method Demonstration</h3>
 <p><b>Method:</b> POST</p>
 <p><b>Endpoint:</b> /login</p>
 <p>Form data would normally be sent in the
 HTTP request body.</p>
 <p><b>Security:</b> Use HTTPS and server-side
 authentication. POST alone does not encrypt data.</p>
 <p>No request was sent to a real server.</p>
 `;
 }
 }
 );
</script>
</body>
</html>
```

Output: -



4. HTML Code Inspection

4.1 Identify Missing Closing Tags

The <h1> heading and <p> paragraph do not have explicit closing tags in the supplied snippet. The <img> element is a void element in HTML and does not require a separate closing tag. Adding explicit end tags for the heading and paragraph makes the intended structure clear.

4.2 Analyze Issues with the <img> Tag

The image element includes a src attribute but no alt attribute. Alternative text is important for screen reader users and can also be shown when the image cannot load. The image path image.jpg must point to an existing, accessible file. Width and height attributes can also help the browser reserve layout space before the image loads.

4.3 Determine How Browsers Handle These Errors

Browsers usually attempt to recover from missing end tags and may still display the heading and paragraph. Nevertheless, relying on automatic error recovery can make the document harder to maintain and may cause unexpected structure or accessibility issues. If image.jpg is missing or unavailable, the image will not display; without alt text, users may not receive a useful textual alternative.

4.4 Corrected HTML Code

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Sample</title>
</head>
<body>
 <h1>Welcome</h1>
 <p>This is a paragraph.</p>
 <img src="image.jpg" alt="Description of the image">
</body>
</html>
```

Explanation:

- The heading and paragraph have explicit closing tags.
- The image includes descriptive alternative text; replace the example with a description of the actual image.
- The document declares HTML5 and includes language, character-set, and viewport metadata.
- The image source should be checked to ensure that the file exists at the specified location.

HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>HTML Code Inspection</title>
 <style>
 * {
 box-sizing: border-box; }  body
{
 margin: 0;
 font-family: Arial, sans-serif;
background: #fef5e7;
 color: #333;}
 header {
 background: #b9770e;
 color: white;
 text-align: center;
 padding: 25px; }
 main {
 max-width: 850px;
 margin: 35px auto;
 padding: 25px;
 background: white;
 border-radius: 12px;
 box-shadow: 0 4px 12px #e6b0aa; }  h2 {
 color: #935116; }
 pre {
 background: #273746;
 color: #f8c471;
 padding: 20px;
 border-radius: 8px;
 overflow-x: auto;
 line-height: 1.7; }
 button {
 padding: 12px 20px;
```

```
 border: none;
 background: #b9770e;
 color: white;
 border-radius: 5px;
 cursor: pointer; }
 button:hover {
 background: #784212; }
 .preview {
 padding: 20px;
 border: 1px solid #ddd;
 border-radius: 8px;
 margin-top: 20px; }
 .result {
 display: none;
 margin-top: 20px;
 padding: 18px;
 background: #fcf3cf;
 border-left: 5px solid #d4ac0d;  line-
height: 1.8; }
 footer {
 text-align: center;
 padding: 20px;
 }
 </style>
</head>
<body>
<header>
 <h1>HTML Code Inspection</h1>
<p>Identify and Correct HTML Errors</p>
</header>
<main>
 <h2>Original HTML Code</h2>
<pre>&lt;!DOCTYPE html&gt;
&lt;html&gt;
&lt;head&gt;
 &lt;title&gt;Sample&lt;/title&gt;
&lt;/head&gt;
```

<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html></pre>
<button onclick="inspectCode()">Inspect HTML</button>

<h2>Corrected Browser Preview</h2>

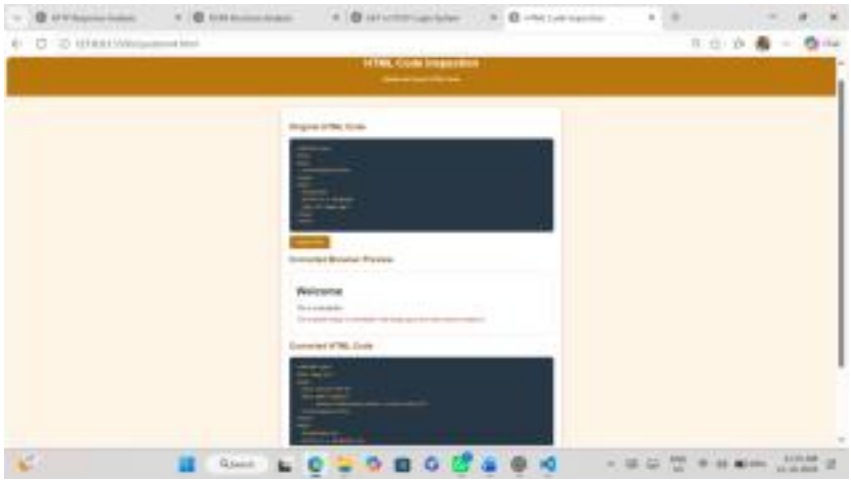
<h1>Welcome</h1>
This is a paragraph.
<img
src="image.jpg"
alt="Example image placeholder"
width="250"
height="120"
onerror="this.style.display='none';
document.getElementById('imageMessage').style.display='block';" >

The example image is unavailable. Add image.jpg to the same
folder to display it.

<h2>Corrected HTML Code</h2>
<pre><!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport"
content="width=device-width, initial-scale=1.0">
<title>Sample</title>
</head>
<body>
<h1>Welcome</h1>
<p>This is a paragraph.</p>
<img src="image.jpg"

```
 alt="Description of the image"&gt;
&lt;/body&gt;
&lt;/html&gt;</pre>
</main>
<footer>
 <p>SIMATS ENGINEERING | Internet Programming</p>
</footer>
<script>
 function inspectCode() {
 const result = document.getElementById("result");
result.style.display = "block";
 result.innerHTML = `
 <h3>HTML Inspection Results</h3>
 <p><b>1. Missing closing tags:</b>
 The original heading and paragraph lack explicit
closing tags.</p>
 <p><b>2. Image issue:</b>
 The image does not contain an alt attribute.</p>
<p><b>3. Browser handling:</b>
 Browsers attempt to recover from malformed HTML.  A missing
image file will prevent the image from loading.</p>  <p><b>4.
Suggested corrections:</b>
 Close the heading and paragraph tags, add descriptive
alternative text, and ensure the image path is correct.</p>  `;
 }
</script>
</body>
</html>
```

Output: -



5. Browser–Server Communication Flow

5.1 Interpret Each Step in the Communication Flow

User enters a URL: The browser parses the URL to identify the scheme, host, port if specified, and requested path.

DNS lookup: The browser or operating system checks available caches and, if needed, asks a DNS resolver to find an IP address for the hostname.

HTTP request: The browser establishes the required connection and sends an HTTP request. HTTPS also includes a TLS security handshake when a secure connection is not already available.

Server processing: The server receives the request, routes it to the appropriate application or resource, and performs any required processing.

Response returned: The server sends an HTTP status code, response headers, and usually a response body. The response indicates whether the request succeeded or requires another action.

Browser renders page: The browser parses HTML, processes CSS, executes applicable JavaScript, fetches referenced resources, and displays the resulting page.

5.2 Identify Where Delays Can Occur

- DNS resolution may take time if the result is not cached or the resolver is slow.
- Connection setup and the TLS handshake can add latency.
- Network congestion, packet loss, or a slow connection can delay request and response transfer.
- Server processing, database queries, or overloaded application services can delay the response.

• Large HTML, CSS, JavaScript, image, and font files can slow downloading and rendering. •

Render-blocking resources and expensive JavaScript can delay when the page becomes usable.

5.3 Analyze the Role of DNS

The Domain Name System (DNS) translates a human-readable hostname, such as www.example.com, into an IP address that the browser can use to connect to the server. DNS caching can avoid repeated lookups. DNS does not normally transfer the webpage itself; it helps the client discover the network address needed to make the connection.

5.4 Suggest Optimizations to Improve Performance

27. Use effective DNS caching and a reliable DNS provider.
28. Use HTTPS with modern protocol support and connection reuse.
29. Compress text resources with Brotli or gzip and optimize image formats and dimensions.
30. Minify and defer non-critical JavaScript; avoid unnecessary render-blocking resources.
31. Use browser caching and a content delivery network (CDN) for suitable static assets. 32. Optimize server code and database queries, and monitor response times. 33. Use responsive images and lazy-load off-screen images where appropriate. 34. Measure real performance with browser developer tools and web performance metrics before and after changes.

HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Browser-Server Communication</title>

 <style>
 * {
 box-sizing: border-box;
 }

 body {
 margin: 0;
 font-family: Arial, sans-serif;
 background: #ebf5fb;
 color: #2c3e50;
```

```
}

header {
background: #1f618d;
color: white;
text-align: center;
padding: 25px;
}

main {
max-width: 900px;
margin: 35px auto;
padding: 25px;
background: white;
border-radius: 12px;
box-shadow: 0 4px 12px #aed6f1;  }

h2 {
color: #1f618d;
}

.step {
padding: 16px;
margin: 12px 0;
background: #d6eaf8;
border-left: 5px solid #2874a6;
border-radius: 6px;
cursor: pointer;
transition: 0.2s;
}

.step:hover {
background: #aed6f1;
transform: translateX(4px);  }

.step.active {
background: #d5f5e3;
```

```
 border-left-color: #239b56;  }

 .description {
 display: none;
 padding: 15px;
 margin-top: 8px;
 background: #f8f9f9;
 border-radius: 6px;
line-height: 1.7;
 }

 .description.show {
 display: block;
 }

 footer {
 text-align: center;
 padding: 20px;
 }
 </style>
</head>

<body>

<header>
 <h1>Browser–Server Communication</h1>
 <p>Interactive Internet Programming Demonstration</p>
</header>

<main>

 <h2>Communication Process</h2>

 <p>Click each step to understand how a browser communicates
with a web server.</p>

 <div class="step" onclick="showStep(0)">
```

Step 1: User Enters a URL

The browser reads the URL to identify the website hostname and requested resource.

Step 2: DNS Lookup

DNS helps translate the domain name into an IP address so the browser can locate the server.

Step 3: HTTP Request

The browser establishes a connection and sends an HTTP request. HTTPS additionally protects communication using TLS.

Step 4: Server Processing

The server receives the request, processes it, and retrieves or generates the required resource.

Step 5: HTTP Response

The server returns a status code, response headers, and usually a response body.

```
 <div class="step" onclick="showStep(5)">
 <b>Step 6: Browser Renders the Page</b>
 <div class="description">
 The browser processes HTML, CSS, JavaScript, and  other
resources to display the webpage.
 </div>
 </div>

 <h2>Performance Optimization</h2>

 <ul>
 <li>Use DNS caching to reduce repeated lookups.</li>
<li>Compress files to reduce download sizes.</li>
<li>Optimize database queries and server processing.</li>
<li>Use browser caching for suitable resources.</li>
<li>Optimize images and avoid unnecessary scripts.</li>
<li>Use a content delivery network when appropriate.</li>
</ul>

</main>

<footer>
 <p>SIMATS ENGINEERING | Internet Programming</p>
</footer>

<script>
 function showStep(index) {

 const steps = document.querySelectorAll(".step");

steps.forEach((step, i) => {

 step.classList.remove("active");

 const description =
 step.querySelector(".description");
```

```
description.classList.remove("show");
if (i === index) {

step.classList.add("active");

description.classList.add("show");
}
});
}
</script>

</body>
</html>
```

Output: -

